

EDS Lab Assignments

Practical 4

Name: Varad Chavan

PRN:202501120024

4.1.1 Pandas – Series Creation and Manipulation

Problem Statement:

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

The usershould enter a list of numbers separated by space when prompted.

Output Format:

Theprogram should display the mean of even and odd numbers separately.
Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Code:

```
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
series = pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

Output:

CODETANTRA

Home

Learn Anywhere

202501040049@mitaoe.ac.in

Support

Logout

4.1.1. Pandas - series creation and manipulation

09:35

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

seriesMa...

SUBMIT

1

import pandas as pd

2

3

Take inputs from the user to create a list of numbers

numbers = list(map(int, input().split()))

4

5

6

Create a Pandas series from the list of numbers

series = pd.Series(numbers)

7

8

Grouping by even and odd numbers and calculating the mean

grouped = series.groupby(series%2==0).mean()

9

10

11

Display the mean of even and odd numbers with labels

grouped.index = ['Even' if is_even else 'Odd' for is_even in

12

grouped.index]

13

print("Mean of even and odd numbers:")

Average time

0.010 s

10.17 ms

Maximum time

0.021 s

21.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1

DEBUG

Expected output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers::

Odd :: 5.0

Even :: 6.0

dtype: float64

Actual output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers::

Odd :: 5.0

Even :: 6.0

dtype: float64

Terminal

Test cases

< PREV

RESET

SUBMIT

NEXT >

4.1.2 Dictionary to Dataframe

Problem Statement:

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).

- Add the new row to the DataFrame.

- Display the DataFrame after adding the new row

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.

- Display the DataFrame after modifying the row

Delete a row:

- Take the row index to be deleted from the user.

- Remove the specified row.

- Display the DataFrame after deleting the row

Add a new column:

- Add a column Gender with values taken from the user.

- Display the DataFrame after adding the new column

Modify a column:

- Convert names to uppercase.

- Display the DataFrame after modifying the column.

Delete a column:

- Remove the Age column.

- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {
```

```
    'Name': ['Alice', 'Bob', 'Charlie'],
```

```
    'Age': [25, 30, 35],
```

```
}
```

```

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new
Name = input("New name: ")
Age = int(input("New age: "))
new = {'Name':Name, 'Age':Age}
df.loc[len(df)] = new
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
index = int(input("Index of row to modify: "))
age = int(input("New age: "))
df.at[index, 'Age']=age
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

# Deleting a row
delt = int(input("Index of row to delete: "))
df = df.drop(delt).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

# Adding a new column
gen = input("Enter genders separated by space: ").split()
df['Gender']=gen
# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
df['Name']=df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")

```

```
print(df)
```

```
# Deleting a column
```

```
df = df.drop(columns = ['Age'])\
```

```
# Display the DataFrame after deleting a column
```

```
print("After deleting a column:")
```

```
print(df)
```

Output:

CODETANTRA Home Learn Anywhere 202501040049@mitaoe.ac.in Support Logout

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Test Results:

- Average time: 0.065 s
- Maximum time: 0.065 s
- 1 out of 1 shown test case(s) passed
- 1 out of 1 hidden test case(s) passed

Test case 1

Expected output	Actual output
Original DataFrame:Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 New name: Susan New age: 29	Original DataFrame:Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 New name: Susan New age: 29
After adding a row:Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 3 Susan 29 Index of row to modify: 3 New age: 27	After adding a row:Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 3 Susan 29 Index of row to modify: 3 New age: 27
After modifying a row:Name Age 0 Alice 27 1 Bob 30 2 Charlie 35 3 Susan 29 Index of row to delete: 3	After modifying a row:Name Age 0 Alice 27 1 Bob 30 2 Charlie 35 3 Susan 29 Index of row to delete: 3

< PREV RESET SUBMIT NEXT >

4.1.3 Student Information

Problem Statement:

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.

- Calculate the average age of the students (limit the average age up to 2 decimal places).

- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Code:

```
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
# write your code here..
print("First five rows:")
print(data.head())
average_age = round(data["Age"].mean(), 2)
print(f"Average age: {average_age}")
filtered_students = data[data["Grade"] <= "B"]
print("Students with a grade up to B")
print(filtered_students)
```

Output:

The screenshot shows the CodeTANTRA IDE interface. On the left, the problem statement for '4.1.3 Student Information' is displayed, including the tasks and sample test cases. The right pane shows the code execution results, indicating that the code passed all test cases. The output of the code is visible, showing the first five rows of the data frame, the average age, and the filtered students with a grade up to B.

Sample Test Cases:

Index	Name	Age	Grade
0	John	25	A
1	Allin	22	B
2	Emma	24	A
3	Mike	23	C
4	Sony	21	B
5	Aalia	26	A

Expected output:

```
First five rows:
  Name Age Grade
0  John  25    A
1  Allin  22    B
2  Emma  24    A
3  Mike  23    C
4  Sony  21    B
Average age: 23.29
Students with a grade up to B:
  Name Age Grade
1  Allin  22    B
2  Emma  24    A
4  Sony  21    B
```

Actual output:

```
First five rows:
  Name Age Grade
0  John  25    A
1  Allin  22    B
2  Emma  24    A
3  Mike  23    C
4  Sony  21    B
Average age: 23.29
Students with a grade up to B:
  Name Age Grade
1  Allin  22    B
2  Emma  24    A
4  Sony  21    B
5  Aalia  26    A
```

4.2.1 Month with the Highest Total Sales

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.
Group the data by Month and calculate the total sales for each month.
Find the month with the highest total sales and display it.
Also, display the total sales for the best month.

Code:

```
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
df['Date'] = pd.to_datetime(df['Date'])
df['Total_Sales_Amount'] = df['Quantity'] * df['Price']
Monthly_Sales = df.groupby(df['Date'].dt.strftime('%Y-%m'))['Total_Sales_Amount'].sum()

# Find the month with the highest total sales
best_month = Monthly_Sales.idxmax()
highest_sales = Monthly_Sales.max()
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

Output:

The screenshot shows a Code Editor interface with a dark theme. The left pane displays the problem statement and sample data. The right pane shows the Python code being executed, with line numbers 1 through 20. The output of the code is displayed at the bottom of the right pane, showing the best month and total sales.

Sample Data:

Date	Product	Quantity	Price	City
2023-01-01	Product A	10	10	New York
2023-01-01	Product A	10	10	Los Angeles
2023-01-01	Product A	10	10	New York
2023-01-01	Product C	10	10	Chicago
2023-01-01	Product A	10	10	Chicago
2023-01-01	Product A	10	10	Los Angeles
2023-01-01	Product C	10	10	New York
2023-01-01	Product A	10	10	Los Angeles
2023-01-01	Product C	10	10	Chicago
2023-01-01	Product C	10	10	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test Case 1

Expected output	Actual output
Best month: 2023-01	Best month: 2023-01
Total sales: \$1230.00	Total sales: \$1230.00

4.2.2 Best Selling Product

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.
Find the product that sold the most in terms of quantity sold.
Display the product that sold the most and the total quantity sold for that product.

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
product_sales = df.groupby("Product")["Quantity"].sum()
# Find the product with the highest total quantity sold
best_product = product_sales.idxmax()
highest_quantity = product_sales.max()
# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

Output:

The screenshot displays a coding environment with the following components:

- Problem Statement:** Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
 - The CSV file contains the columns: Date, Product, Quantity, Price, and City.
 - Find the product that sold the most in terms of quantity sold.
 - Display the product that sold the most and the total quantity sold for that product.
- Sample Data:**

```
Date,Product,Quantity,Price,City
2021-01-01,Product A,10,New York
2021-01-01,Product A,10,Los Angeles
2021-01-01,Product A,10,New York
2021-01-01,Product C,5,Chicago
2021-01-01,Product A,10,Chicago
2021-01-01,Product A,10,Los Angeles
2021-01-01,Product C,5,New York
2021-01-01,Product A,10,Los Angeles
2021-01-01,Product A,10,Chicago
2021-01-01,Product C,5,Los Angeles
```
- Note:** The data cannot be displayed in the file. You can refer to the sample data provided for insights.
- Sample Test Cases:** A section for testing the solution.
- Code Editor:** Contains the Python code for finding the best-selling product.
- Test Results:** Shows 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The output for the test case is:

```
Best selling product: Product A
Total quantity sold: 23
```
- Navigation:** Buttons for PREVIOUS, RESET, SUBMIT, and NEXT.

4.2.3 City that Sold the Most Products

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
# write the code..
```

```
city_sales = df.groupby("City")["Quantity"].sum()
```

```
best_city = city_sales.idxmax()
```

```
# Display the result
```

```
print(f"City sold the most products: {best_city}")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '4.2.3. City that Sold the Most Products' is shown, including sample data and a note. The main editor contains the Python code for solving the problem. The right sidebar shows the execution results, including test case status and output.

Sample Data:

Date	Product	Quantity	Price	City
2023-01-01	Product A	10	100	New York
2023-01-01	Product A	15	150	Los Angeles
2023-01-02	Product A	20	200	New York
2023-01-02	Product C	5	50	Chicago
2023-01-02	Product A	25	250	Los Angeles
2023-01-03	Product C	10	100	New York
2023-01-03	Product A	30	300	Los Angeles
2023-01-03	Product A	35	350	Chicago
2023-01-03	Product C	15	150	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Test Cases:

Test Case	Status	Expected Output	Actual Output
Test case 1	Passed	City sold the most products: Los Angeles	City sold the most products: Los Angeles
Test case 2	Passed	City sold the most products: Los Angeles	City sold the most products: Los Angeles

4.2.4 Most Frequently Sold Product Pairs

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the following columns: Date, Product, Quantity, Price, and City.

For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).

Output the product pair/s that was sold most frequently.

Code:

```
import pandas as pd
```

```
from itertools import combinations
```

```
from collections import Counter
```

```
# Prompt user to input the file name
```

```
file_name = input()
```

```
# Read data from the specified CSV file
```

```
df = pd.read_csv(file_name)
```

```
# write the code
```

```
grouped = df.groupby("Date")["Product"].apply(list)
```

```
pairs_counter = Counter()
```

```
for products in grouped:
```

```
    pairs = combinations(sorted(products), 2)
```

```
    pairs_counter.update(pairs)
```

```
max_frequency = max(pairs_counter.values())
```

```
most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq ==  
max_frequency]
```

```
# Output the most frequent product pairs
```

```
for (product1, product2), frequency in most_common_pairs:
```

```
    print(f"{product1} and {product2}: {frequency} times")
```

Output:

CODETANTRA

Home Learn Anywhere

202501040049@mitaoe.ac.in Support Logout

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pairs that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	2	10	New York
2025-01-01	Product B	1	15	Los Angeles
2025-01-01	Product C	1	10	New York
2025-01-01	Product C	2	20	Chicago
2025-01-01	Product A	2	10	Chicago
2025-01-01	Product A	2	10	Los Angeles
2025-01-01	Product C	2	20	New York
2025-01-01	Product A	2	10	Los Angeles
2025-01-01	Product A	2	10	Chicago
2025-01-01	Product C	10	10	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- Product A and Product B (2 times)
- Product B and Product C (2 times)

request...

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 # write the code
12 grouped = df.groupby("Date")["Product"].apply(list)
13
14 pairs_counter = Counter()
15 for products in grouped:
16     pairs = combinations(sorted(products), 2)
17     pairs_counter.update(pairs)
18
```

Average time: 0.218 s

Maximum time: 0.028 s

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Actual output

Product A and Product B: 2 times

Product A and Product B: 2 times

Product A and Product C: 2 times

Product A and Product C: 2 times

Terminated

Test cases

PREV

RESET

SUBMIT

NEXT

4.2.5 Titanic Dataset Analysis and Data Cleaning

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
print(data.head())
```

```
# 1. Display the first 5 rows of the dataset
print(data.tail())
```

```
# 2. Display the last 5 rows of the dataset
print(data.shape)
```

```
# 3. Get the shape of the dataset
print(data.info())
```

```
# 4. Get a summary of the dataset (info)
print(data.describe())
```

```
# 5. Get basic statistics of the dataset
print(data.isnull().sum())
```

```
# 6. Check for missing value
median_age = data['Age'].median()
data['Age'].fillna(median_age, inplace = True)
```

```
# 7. Fill missing values in the 'Age' column with the median age
mode_embarked = data['Embarked'].mode() [0]
data['Embarked'].fillna(mode_embarked, inplace = True)
```

```
# 8. Fill missing values in the 'Embarked' column with the mode
data.drop('Cabin', axis=1, inplace = True)
```

```
# 9. Drop the 'Cabin' column due to many missing values
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
```

Output:

The screenshot displays a Jupyter Notebook environment with a dark theme. The top navigation bar includes the 'CDETANTRA' logo, a home link, a 'Learn Anywhere' link, a user profile '202501040049@mitaoe.ac.in', a 'Support' link, and a 'Logout' button. The notebook title is '4.2.5. Titanic Dataset Analysis and Data Cleaning'. The main text area contains a series of 10 numbered tasks for data analysis and cleaning. Below the tasks, a table lists the columns of the Titanic dataset: PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, and Embarked. A 'Sample Data:' section shows a snippet of the dataset's raw data. The right sidebar shows the 'Test case 1' results, indicating that 1 out of 1 shown test case(s) passed. The 'Expected output' and 'Actual output' sections show the same data snippet, confirming the test passed. At the bottom, there are buttons for 'Terminal', 'Test cases', and navigation controls: 'PREV', 'RESET', 'SUBMIT', and 'NEXT'.

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

- Display the first 5 rows of the dataset.
- Display the last 5 rows of the dataset.
- Get the shape of the dataset (number of rows and columns).
- Get a summary of the dataset (using `.info()`).
- Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
- Check for missing values and display the count of missing values for each column.
- Fill missing values in the 'Age' column with the median age.
- Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
- Drop the 'Cabin' column due to many missing values.
- Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Owen Harris	male	32	0	0	21517	7.25	5	S
2	1	1	Mr. John Bradley (Florence Briggs Thayer)	female	36	1	0	PC 17599	71.2833	C85	C
3	1	3	Mr. William Henry	male	36	0	0	21012	53.1000	C123	S
4	0	3	Mr. James	male	26	0	0	11380	51.0000	Q	S

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. Owen Harris, male, 32, 0, 0, 21517, 7.25, 5, S
2, 1, 1, Mr. John Bradley (Florence Briggs Thayer), female, 36, 1, 0, PC 17599, 71.2833, C85, C
3, 1, 3, Mr. William Henry, male, 36, 0, 0, 21012, 53.1000, C123, S
4, 0, 3, Mr. James, male, 26, 0, 0, 11380, 51.0000, Q, S
```

Test case 1: 1 out of 1 shown test case(s) passed

Expected output: PassengerId Survived Pclass ... Fare Cabin Embarked

Actual output: PassengerId Survived Pclass ... Fare Cabin Embarked

5 rows x 12 columns

Terminal Test cases

PREV RESET SUBMIT NEXT

4.2.6 Titanic Dataset Analysis and Data Cleaning-2

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['Alone'] = np.where(data['FamilySize'] == 0, 1, 0)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns = ['Embarked'], drop_first = True)
```

```
# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print(mean_age)
```

```
# 5. Get the median fare of passengers
```

```
median_fare = data['Fare'].median()
print(median_fare)
```

```
# 6. Get the number of passengers by class
passengers_by_class = data['Pclass'].value_counts()
print(passengers_by_class)
```

```
# 7. Get the number of passengers by gender
passengers_by_gender = data['Sex'].value_counts().sort_index()
print(passengers_by_gender)
```

```
# 8. Get the number of passengers by survival status
passengers_by_survival = data['Survived'].value_counts().sort_index()
print(passengers_by_survival)
```

```
# 9. Calculate the survival rate
survival_rate = data['Survived'].mean()
print(survival_rate)
```

```
# 10. Calculate the survival rate by gender
survival_rate_by_gender = data.groupby('Sex')['Survived'].mean()
print(survival_rate_by_gender)
```

Output:

The screenshot shows a Jupyter Notebook titled "4.2.6. Titanic Dataset Analysis and Data Cleaning - 2". The notebook contains Python code for analyzing the Titanic dataset. The code includes imports for pandas and numpy, loading the dataset from a CSV file, and performing various calculations as specified in the tasks.

The notebook also displays the Titanic dataset columns and a sample of data. The columns are: PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, and Embarked. The sample data shows the first few rows of the dataset.

On the right side of the notebook, the test results are displayed. The tests passed, and the output for the first test case is shown. The output includes the expected and actual values for the survival rate and the survival rate by gender.

4.2.7 Titanic Dataset Analysis and Data Cleaning-3

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Calculate the survival rate by class
```

```
print(data.groupby('Pclass')['Survived'].mean())
```

```
# 2. Calculate the survival rate by embarked location
```

```
print(data.groupby('Embarked_S')['Survived'].mean())
```

```
# 3. Calculate the survival rate by family size
```

```
print(data.groupby('FamilySize')['Survived'].mean())
```

```
# 4. Calculate the survival rate by being alone
```

```
print(data.groupby('IsAlone')['Survived'].mean())
```

```
# 5. Get the average fare by class
```

```
print(data.groupby('Pclass')['Fare'].mean())
```

6. Get the average age by class

```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

Output:

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Bruce Harris	male	21.0	1	0	2101	53.1	5	
2	1	1	Mr. John Bradley	female	19.0	0	0	PC 17596	71.283	C85	C
3	1	3	Mr. James Brown	female	18.0	0	0	STON/O2	53.1	5	
4	1	3	Mr. James Brown	female	18.0	0	0	STON/O2	53.1	5	
5	1	3	Mr. James Brown	female	18.0	0	0	STON/O2	53.1	5	
6	1	3	Mr. James Brown	female	18.0	0	0	STON/O2	53.1	5	
7	1	3	Mr. James Brown	female	18.0	0	0	STON/O2	53.1	5	
8	1	3	Mr. James Brown	female	18.0	0	0	STON/O2	53.1	5	
9	1	3	Mr. James Brown	female	18.0	0	0	STON/O2	53.1	5	
10	1	3	Mr. James Brown	female	18.0	0	0	STON/O2	53.1	5	

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
```

Test case 1: 1 out of 1 shown test case(s) passed

Expected output	Actual output
Pclass:	Pclass:
1 ... 0.429638	1 ... 0.429638
2 ... 0.472826	2 ... 0.472826
3 ... 0.242363	3 ... 0.242363
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S:	Embarked_S:
0 ... 0.586873	0 ... 0.586873
1 ... 0.330957	1 ... 0.330957
Name: Survived, dtype: float64	Name: Survived, dtype: float64
FamilySize:	FamilySize:
0 ... 0.383538	0 ... 0.383538
1 ... 0.552795	1 ... 0.552795
2 ... 0.578431	2 ... 0.578431
3 ... 0.724138	3 ... 0.724138
4 ... 0.200000	4 ... 0.200000
5 ... 0.136364	5 ... 0.136364
6 ... 0.333333	6 ... 0.333333
7 ... 0.000000	7 ... 0.000000
8 ... 0.000000	8 ... 0.000000
Name: Survived, dtype: float64	Name: Survived, dtype: float64
IsAlone:	IsAlone:
0 ... 0.586873	0 ... 0.586873
1 ... 0.383538	1 ... 0.383538
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Pclass:	Pclass:
1 ... 0.429638	1 ... 0.429638

4.2.8 Titanic Dataset Analysis and Data Cleaning-4

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Get the number of survivors by gender
```

```
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
print(survivors_by_gender)
```

```
# 2. Get the number of non-survivors by gender
```

```
non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()
print(non_survivors_by_gender)
```

```
# 3. Get the number of survivors by embarked location
```

```
survivors_by_embarked_s = data[data['Survived'] == 1]['Embarked_S'].value_counts()
print(survivors_by_embarked_s)
```

```
# 4. Get the number of non-survivors by embarked location
```

```
non_survivors_by_embarked_s = data[data['Survived'] == 0]['Embarked_S'].value_counts()
```

```
print(non_survivors_by_embarked_s)
```

```
# 5. Calculate the percentage of children (Age < 18) who survived
```

```
children = data[data['Age'] < 18]
```

```
children_survival_rate = children['Survived'].mean()
```

```
print(children_survival_rate)
```

```
# 6. Calculate the percentage of adults (Age >= 18) who survived
```

```
adults = data[data['Age'] >= 18]
```

```
adults_survival_rate = adults['Survived'].mean()
```

```
print(adults_survival_rate)
```

```
# 7. Get the median age of survivors
```

```
median_age_survivors = data[data['Survived'] == 1]['Age'].median()
```

```
print(median_age_survivors)
```

```
# 8. Get the median age of non-survivors
```

```
median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()
```

```
print(median_age_non_survivors)
```

```
# 9. Get the median fare of survivors
```

```
median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()
```

```
print(median_fare_survivors)
```

```
# 10. Get the median fare of non-survivors
```

```
median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()
```

```
print(median_fare_non_survivors)
```

Output:

CODETANTRA

Home

Learn Anywhere

202501040009@mitaoe.ac.inSupportLogout

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

100%

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Bruce Harris	male	21.0	1	0	21517	7.25	5	
2	1	1	Mr. John Bradley (Florence Briggs Thayer)	female	38.0	1	0	PC 17599	53.00	C85	C
3	1	3	Mr. James (Hector) Gibson	male	35.0	1	0	3101	53.00	5	
4	1	3	Mr. James (Hector) Gibson	male	35.0	1	0	3101	53.00	5	
5	1	3	Mr. James (Hector) Gibson	male	35.0	1	0	3101	53.00	5	
6	1	3	Mr. James (Hector) Gibson	male	35.0	1	0	3101	53.00	5	
7	1	3	Mr. James (Hector) Gibson	male	35.0	1	0	3101	53.00	5	
8	1	3	Mr. James (Hector) Gibson	male	35.0	1	0	3101	53.00	5	
9	1	3	Mr. James (Hector) Gibson	male	35.0	1	0	3101	53.00	5	
10	1	3	Mr. James (Hector) Gibson	male	35.0	1	0	3101	53.00	5	

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. Bruce Harris, male, 21.0, 1, 0, 21517, 7.25, 5, 
2, 1, 1, Mr. John Bradley (Florence Briggs Thayer), female, 38.0, 1, 0, PC 17599, 53.00, C85, C
3, 1, 3, Mr. James (Hector) Gibson, male, 35.0, 1, 0, 3101, 53.00, 5, 
4, 1, 3, Mr. James (Hector) Gibson, male, 35.0, 1, 0, 3101, 53.00, 5, 
5, 1, 3, Mr. James (Hector) Gibson, male, 35.0, 1, 0, 3101, 53.00, 5, 
6, 1, 3, Mr. James (Hector) Gibson, male, 35.0, 1, 0, 3101, 53.00, 5, 
7, 1, 3, Mr. James (Hector) Gibson, male, 35.0, 1, 0, 3101, 53.00, 5, 
8, 1, 3, Mr. James (Hector) Gibson, male, 35.0, 1, 0, 3101, 53.00, 5, 
9, 1, 3, Mr. James (Hector) Gibson, male, 35.0, 1, 0, 3101, 53.00, 5, 
10, 1, 3, Mr. James (Hector) Gibson, male, 35.0, 1, 0, 3101, 53.00, 5, 
```

ExpanderTitanicDat...

1import pandas as pd

2import numpy as np

3

4Load the Titanic dataset

Average time: 0.049 sMaximum time: 0.049 s1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

female ... 233

male ... 189

Name: Sex, dtype: int64

male ... 468

female ... 81

Name: Sex, dtype: int64

1 ... 217

0 ... 125

Name: Embarked_S, dtype: int64

1 ... 427

0 ... 122

Name: Embarked_S, dtype: int64

0.539230088495575

0.381815119767955

28.0

28.0

26.0

18.5

Terminal

Test cases

PREV

RESET

SUBMIT

NEXT